AlgaMask: RAG implementation sprint

Revision 1.1 | 22 October 2026 | Task-based execution plan with implementation marks

Outcome

Build a Flutter application that retrieves reviewed scientific evidence, uses it in a six-role formulation workflow, and produces traceable bar concepts, calculated nutrition and a report. Complete a physical B0/A0/A1 comparison before claiming measured sensory improvement.

The seven stages retain the original sprint's Day 1-7 labels for comparison. They are work packages, not calendar deadlines. Finish each acceptance gate before advancing dependent work. This revised plan governs future execution; the original PDF remains the requirements reference.

Starting position

- Flutter and FastAPI skeletons exist. The main workflow has six mock roles; the Day 1 preview is also a placeholder path.
- The reviewed-table directory contains 286 extracted rows: 30 compounds, 45 algae observations, 20 ingredient forms, 19 nutrition records and 22 evidence chunks. None are approved.
- A separate extraction packet registers 15 selected sources and adds 25 study records. Full-text review is incomplete for some sources.
- Eight draft retrieval questions exist. There is no built vector index, operational embedding provider, evaluated retrieval service or published dataset version.
- Molecular features and draft formulations exist, but the live formulation workflow still uses demonstration logic. Physical prototype and sensory results are absent.

Product and RAG decisions

- Required proof path: one 45 g spirulina bar, using the current oats, dried dates and unsweetened cocoa baseline. Freeze exact proportions in Stage 1. Keep Chlorella as an alternative path only when its evidence and ingredient records satisfy the same gates.
- B0 contains no algae. A0 adds a specified spirulina dose with an explicit replacement rule. A1 retains A0's dose and adds a documented intervention. Existing 2% and 5% options are project screening choices pending dose review.
- Index reviewer-written evidence chunks. Keep compound identities, nutrition, allergens and numeric bounds in structured tables for deterministic lookup and calculation.
- Use one pinned local sentence-embedding model and one local FAISS index. Confirm model quality and runtime compatibility before locking their versions. Generation uses one configured model adapter shared by the agents.
- Keep retrieval similarity, scientific evidence strength and claim status as separate fields. A high similarity score does not establish efficacy.

Sequence and ownership

Stage 1 contracts → Stage 2 approved corpus and evaluated retrieval → Stage 3 grounded formulation → Stage 4 generated, cited application output → Stage 5 physical validation → Stage 6 release testing → Stage 7 handoff.

Assign a named owner and reviewer to each stage before starting it. One person may own multiple stages. Dataset and index approval decisions must record who made them; a validation command does not grant approval.

Completed work: where RAG fits

Status marks: **DONE** means the named artifact exists. **PARTIAL** means it needs further work. **RAG ADD** means create a missing component. **RAG REPLACE** means replace placeholder behavior. These marks describe the inspected local workspace, including uncommitted files; they do not mean every artifact is scientifically approved or released.

Work already completed and reusable

- **DONE - Source discovery and extraction.** The 15-source register and 25 new study records are in data/extractions/day2_sources.json. Reuse them for corpus preparation; some findings still require full-text verification.
- **DONE - Structured dataset and validation.** The 21 reviewed tables contain 286 extracted rows. tools/validate.py and tools/validate_extractions.py pass structural checks. Keep these checks; extend them to enforce publication eligibility and approved reference chains.
- **DONE - Chemistry data extraction.** Thirty compounds, 45 algae observations and existing molecular feature rows provide reusable inputs. Their review and runtime scoring remain partial.
- **PARTIAL - RAG corpus.** Twenty-two evidence chunks exist in data/reviewed/evidence_chunks.csv. None are approved. Review and extend these notes rather than starting the research again.
- **PARTIAL - Ingredients and nutrition.** Twenty ingredient forms and 19 nutrition records exist, with only one provisional screening bound. Finish exact-form matching and bounds for the selected recipes.
- **DONE - Runnable application skeleton.** Flutter forms, FastAPI endpoints, six role functions, a trace, recipe concepts and report scaffolding exist. They demonstrate application wiring, not completed RAG.
- **PARTIAL - Retrieval evaluation.** Eight draft questions exist in data/reviewed/retrieval_evaluations.csv. Reuse and review them, add held-out cases and run against the real index.

Exact connection points in the existing flow

Current /runs path: request → Profile → Retrieval → Chemistry → Recipe → Critic → Impact → response. Keep this structure and replace the relevant placeholder behavior at each handoff.

- **RAG REPLACE - Retrieval.** agents/retrieve.py currently ignores request details and calls retrieve_mock(). This is the first live-workflow insertion point for real evidence search.
- **RAG REPLACE - Chemistry.** agents/chemistry.py receives evidence but selects the requested flavour and copies all chunk IDs. Make it evaluate relevant support and retain only the evidence actually used.
- **RAG REPLACE - Recipe.** agents/recipe.py currently ignores candidate content and calls the mock generator. Make supported candidates and structured bounds drive recipe generation.
- **RAG ADD - Critic grounding.** agents/critic.py currently checks recipe validity. Add checks that citations exist in the retrieved context and that the proposed claim is supported.

Completion boundary: the existing skeleton and extraction work are reusable. The reviewed corpus, embedding/index pipeline, real search and grounded generation are still implementation work. See the next page for the exact file changes.

RAG marks: files to change next

Data and index: Stage 2

- **RAG REPLACE - Approval gate.** In services/algamask/retrieval.py, approved_chunks() checks only status and source ID. Extend eligibility checks to required provenance, source/reference approval, permitted content and the selected release.
- **RAG ADD - Embedding implementation.** services/algamask/embeddings.py contains only a protocol. Add the actual model adapter, normalization, batching and pinned configuration.
- **RAG ADD - Index builder.** Create tools/build_rag_index.py. Write vectors, metadata and a manifest under indexes/reviewed_evidence/0.1.0/; the current index directory contains only a README.
- **RAG REPLACE - Search implementation.** Replace retrieve_mock() in services/algamask/retrieval.py with version-checked index loading, query embedding, eligible metadata filtering, ranking and an explicit no-evidence result.
- **RAG ADD - Configuration.** Update pyproject.toml and dependency locks. Add model/revision, index path and search settings in a configuration module. An embedding/index model mismatch must fail explicitly.
- **RAG ADD - Evaluation runner.** Create retrieval tests and a runner for retrieval_evaluations.csv. Save rankings and acceptance results. Populate dataset_versions.csv only after corpus and retrieval gates pass.

Backend workflow and generation: Stages 1, 3 and 4

- **RAG ADD - Typed results.** Extend services/algamask/schemas.py with query/hit/result and cited-claim schemas. Its existing evidence field is an untyped list; RunResult.mode currently permits only mock.
- **RAG REPLACE - Retrieval role.** Update services/algamask/agents/retrieve.py to form diagnosis/intervention queries from the request and pass filters into real search.
- **RAG REPLACE - Supervisor handoff.** Update agents/supervisor.py to pass real hits and versions downstream, record role-specific status and handle insufficient evidence. It currently marks every trace step mock.
- **RAG ADD - Grounded generation.** Add a model/provider adapter and versioned prompts; connect Chemistry, Recipe and Critic as marked on the previous page. There is currently no live generation client.
- **RAG ADD - Inspection API.** Extend services/algamask/api.py with /rag/search and readiness/version fields. Keep /runs as the real application entry point.

Flutter, report and tests: Stages 4 and 6

- **RAG REPLACE - Screen connection.** apps/flutter/lib/src/algamask/api_client.dart currently posts to /day1/preview. Move the active screen to /runs and adapt its parsing/widgets to the returned run contract.
- **RAG REPLACE - Evidence display.** Update the active result widgets and report_view.dart: existing evidence cards describe synthetic sources. Show real passages, source links, exact locations, evidence class and indirect-evidence labels. Verify the upgraded view is actually mounted by the active screen.
- **RAG REPLACE - Export and tests.** Extend services/algamask/report.py to include retrieved citations and release identity. Update mock workflow assertions in tests/algamask/test_workflow.py; add search, grounding and Flutter evidence tests.

Start with corpus review plus embedding/index implementation, then connect the Retrieval role. Finish grounded generation and the active Flutter route before marking end-to-end RAG complete. This annotated plan identifies the work; the application changes themselves have not been implemented by annotating it.

Stage 1 / Day 1: Freeze scope and contracts

Status: skeleton substantially complete; contracts and physical recipe decisions remain.

Tasks

- Reconcile the current Day 1 preview and /runs paths. Select /runs as the eventual application workflow and document how Flutter transitions from the preview.
- Freeze exact B0 ingredient forms and grams, serving size, A0 inclusion, and what ingredient is displaced. Define A1 as the same algae dose plus an intervention. Record batch and portion bases explicitly.
- Confirm five sensory categories: earthy/muddy, grassy/green, marine/fishy, sulfurous and bitter. Primary calculated outcome is protein per 100 kcal; sensory outcomes use the recorded trial scale.
- Define typed RetrievalQuery, EvidenceHit and RetrievalResult contracts. Query fields include purpose, text, species, matrix, related IDs and result limits. Results include hits, applied filters, index version and an explicit no-evidence reason.
- Define claim output as claim text plus cited chunk IDs, evidence status and limitations. Keep retrieved passage text separate from system instructions in model prompts.
- Preserve explicit mock, calculated, predicted, literature-derived, database-derived, measured and unknown labels. A partially upgraded run must describe which components remain mocked.
- Assign the data reviewer, technical owner and prototype owner. List any provider credentials needed for generation; local retrieval must remain independently testable.

Files and deliverables

- Update docs/DAY1_SCOPE.md and data/sprint/scope.json with resolved product decisions.
- Extend services/algamask/schemas.py with RAG and citation contracts.
- Add examples for a valid hit, no-evidence response, indirect evidence and a version mismatch.
- Record the selected runtime, embedding model/revision and generation adapter configuration when confirmed. Store secrets outside the repository.

Acceptance gate

- A reviewer can reproduce B0 and A0 from exact ingredient forms, quantities and the replacement rule.
- A valid request and evidence response pass schema validation; invalid filters and fabricated citation IDs fail validation.
- The app's current mock path remains runnable while the new contracts are introduced.

Dependency and scope rule

Stage 2 corpus review can proceed while recipe proportions are finalized. Stage 3 recipe generation and Stage 5 preparation require the frozen recipe and reviewed dose basis. If the matrix changes, update retrieval filters and evaluation cases before treating earlier results as applicable.

Stage 2A / Day 2: Approve the RAG corpus

Status: source discovery and extraction largely present; scientific review and publication eligibility remain.

Tasks

- Reconcile the 15-source extraction register with sources.csv. Preserve source IDs and exact section/table/page references; resolve the BJDV DOI discrepancy.
- Review existing chunks and promote usable extraction findings into short, independent evidence notes. Each note should cover one claim and retain its material, process, dose basis, panel or analytical method where relevant.
- Approve at least 20 useful chunks. Include both diagnosis and intervention evidence, with coverage for each enabled algae and the demonstrated matrix. Do not create duplicates merely to reach the count.
- Record source permissions and permitted use of each authored summary or quotation. Retain approval author and timestamp. Exclude unresolved claims from the release or explicitly bound their use to what the source supports.
- Resolve the recorded recipe, unit and evidence gaps: fruit-bar mass balance and nutrition labels, cinnamon denominator, ITEPA method/basis, neroli volume-to-mass conversion and processing-only Chlorella claims.
- Complete structured nutrition for every major ingredient in the chosen recipes. Review 20-40 ingredient forms and add applicable, source-backed bounds for every ingredient the optimizer may vary.
- Preserve missing nutrition as unknown. Keep dry-weight values separate from as-sold values. Keep experimental treatment ranges separate from regulatory or project screening limits.

Chunk contract and metadata

- Required fields: chunk_id, authored claim summary, diagnosis/intervention tag, source_id, source location, reviewer, review time and review_status=approved.
- Retrieval metadata: species/material IDs, canonical matrix, ingredient/compound/descriptor IDs, evidence type, support class and limitations. Add an explicit species field or a validated join where the existing material links are insufficient.
- Attach dataset version at publication. Preserve quotation permissions separately from source access: being publicly readable does not by itself establish every reuse permission.
- Validate every referenced record and source. An approved chunk must not silently inherit unresolved identities or claims from an unreviewed linked record.

Deliverables and gate

Deliver reviewed tables, an exclusion/issue log and at least 20 approved chunks with useful coverage. Every released record has complete provenance and correct units. Known unresolved findings remain outside the approved claim set. A source's approval does not automatically approve every finding attributed to it.

The corpus may be prepared for engineering tests earlier using clearly labelled fixtures. The published index must pass this gate before it powers the real evidence path.

Stage 2B / Day 2: Build and evaluate retrieval

Status: **RAG ADD / RAG REPLACE** - interfaces and mock retrieval exist; no operational embedding model, index or evaluated real search exists.

Build the index

- Implement embeddings.py with one local model, batch embedding, fixed revision, fixed text preparation and vector normalization. Verify language quality and compatibility with the selected Python runtime before locking dependencies.
- Add tools/build_rag_index.py. Read approved chunks, join eligible source metadata, reject missing provenance and deduplicate before embedding. Do not silently relabel extracted rows as approved.
- Store vectors.faiss, chunks.json and manifest.json under indexes/reviewed_evidence/0.1.0/. Record ordered chunk IDs, text hashes, model/revision, dimensions, similarity metric, dataset hash, artifact hashes and build configuration.
- Use matching normalization and distance settings at build and query time. Load the index once per application process. Validate manifest/model/dimension compatibility at startup.

Retrieve and expose evidence

- Replace fixture retrieval with diagnosis and intervention searches. Build queries from species, matrix, sensory risks and flavour direction. Apply review/species/tag filters before ranking within eligible candidates; exact scoring over a small eligible subset is acceptable.
- Prefer same-matrix evidence. If configured, allow an explicit related-matrix backoff and label it indirect. Never silently present another species or matrix as direct support.
- Retrieve a candidate pool, apply duplicate/source-diversity rules and return up to five useful hits per purpose. Tune similarity thresholds using development questions; scores are model-specific, not universal cutoffs.
- Return chunk text, citation IDs/location, evidence class, score, applied filters and version. No qualifying hit produces insufficient_data; index/model failures produce a distinct service error.
- Add /rag/search for inspection and RAG readiness/version fields to /health. The generation path will consume these results in Stage 4.

Evaluate and freeze

- Review all eight existing questions. Separate tuning cases from held-out acceptance cases, then add at least five held-out positive cases spanning diagnosis, intervention and enabled species. Record expected chunks/sources and acceptable indirect evidence before running them.
- Require the expected evidence in the top three for every held-out positive case. Add negative queries and filter/version/error tests: no unapproved leakage, fabricated source IDs or silent fallback to fixture cards.
- Save rankings, scores, filters, pass/fail results and inspected failure notes. If a held-out case is used for tuning, replace it with a new held-out case.
- Publish dataset/index 0.1.0 only after review and evaluation pass. Record hashes, model identity, validation report and code commit in dataset_versions.csv. Rebuilds must preserve corpus identity and ranking behavior within documented tolerances; do not assume binary index files are universally byte-identical.

Stage 2 acceptance gate

At least 20 approved chunks; reproducible index; filtered searches with resolvable citations; passing positive and negative retrieval cases; matching version manifest; frozen release record. Embeddings alone do not complete RAG: grounded generation is the Stage 4 gate.

Stage 3 / Day 3: Ground chemistry and recipes

Status: **PARTIAL / RAG REPLACE** - feature and draft recipe rows exist; production scoring and evidence-driven constrained generation remain.

Tasks

- Audit existing molecular features. Store usable fingerprint vectors or regenerate them from the pinned molecular representation; a hash alone cannot support similarity calculations. Fix radius, bit count, chirality, standardization and RDKit version.
- Link species/material observations to compounds, sensory descriptors and retrieved intervention evidence. Keep analytical detection, odor activity, food-matrix sensory evidence and hypotheses distinct.
- Implement explicit, normalized molecular/sensory/evidence score components. Record weights and missing-modality handling. Reduce claim strength when evidence is indirect; do not multiply a vector score into a purported probability of masking.
- Make the Chemistry role consume retrieved hits and structured compound/ingredient tables. Each candidate retains its rationale, source and chunk IDs, evidence class, score components and limitations.
- Generate recipes through a small deterministic constrained search. Use only enabled ingredient forms with reviewed nutrition, allergen information and applicable screening bounds.
- Enforce mass balance, serving size, algae dose/replacement, exclusions, bounds, sugar and calorie constraints. Define how missing mandatory data blocks or reduces the feasible search space.
- Produce three distinct feasible variants when supported. If fewer than three exist, return the actual count and constraint failures rather than inserting unsupported alternatives.
- Preserve random seed, configuration, rejected candidates and dataset/index versions. Compare a fixed input with hand-reviewed expectations.

Files and deliverables

- Implement scoring in molecular.py, optimizer.py and the Chemistry/Recipe roles.
- Update validation to check ingredient IDs, exact forms, bounds, numeric units and evidence links.
- Add deterministic fixtures for valid recipes, incompatible ingredients, excluded allergens, missing bounds and an infeasible search.
- Save candidate score breakdowns and generated recipe records with their evidence lineage.

Acceptance gate

The fixed feasible scenario produces three distinct, reproducible recipes that pass every hard constraint and cite the evidence actually used. Infeasible inputs return explicit reasons. Nutrition and concentration arithmetic come from structured records and deterministic code. A candidate explanation must not imply that molecular similarity proves sensory improvement.

Scope control

Use the reviewed 45 g spirulina path first. Enable Chlorella only after equivalent evidence, ingredient and evaluation coverage exists. An unsupported option remains unavailable with a clear reason. Complex optimization, extra algae and optional carbon scoring are outside the required path.

Stage 4 / Day 4: Complete RAG and the live app

Status: **PARTIAL / RAG ADD / RAG REPLACE** - API, Flutter and six mock roles exist; generation, grounding checks and real evidence handoffs remain.

Generate from retrieved evidence

- Implement Profile, Retrieval, Chemistry, Recipe, Critic and Impact as typed roles coordinated by an explicit supervisor state machine.
- Configure one model adapter for grounded reasoning and explanation. Supply selected evidence with stable chunk IDs, metadata and limitations in delimited data blocks. Treat source content as evidence, never as instructions.
- Require generated claims and candidate rationales to cite supplied chunk IDs. Validate that cited IDs belong to the current retrieval context and resolve to the same frozen release. Validate claim support through the Critic plus reviewed examples; ID validation alone cannot prove entailment.
- Require abstention or a qualified hypothesis for unsupported efficacy claims. Preserve the distinction between measured literature results and project measurements.
- Add one retry for model/schema failures and at most two critique/revision cycles. Persist failures and rejected claims. A deterministic evidence summary may be a labelled fallback; report whether generation was model-assisted or templated.

Connect API, calculations and Flutter

- Replace Retrieval Agent fixture calls with the Stage 2 service. Pass diagnosis and intervention results into Chemistry and Recipe. Update trace states per role instead of marking the whole run complete or real prematurely.
- Connect Flutter to /runs; retire its reliance on the Day 1 preview for the real workflow. Display role progress, generated recipes, evidence, citation locations, indirect-evidence labels and run limitations.
- Calculate B0/A0/A1 nutrition from exact ingredient forms and grams. Report per serving and protein per 100 kcal. Missing required nutrients produce unknown results, not zero.
- Preserve runs and evidence snapshots, provider/model/prompt versions, index version, recipe decisions and calculated outputs. A later dataset update must not rewrite an earlier run's report.
- Surface no evidence, index unavailable, provider failure and infeasible recipe as different states. Never silently replace a failed real path with synthetic evidence.

Files and deliverables

Update schemas.py, api.py, agents/retrieve.py, agents/supervisor.py, downstream roles, nutrition/report logic and the Flutter input/result screens. Add a configuration/provider module and persistent run storage. Keep secrets out of exports and logs.

Acceptance gate

One live request retrieves approved passages, generates a cited explanation, runs the six roles, returns constraint-valid recipes and calculated nutrition, and displays its evidence in Flutter. Citation IDs resolve to the supplied passages; reviewed examples demonstrate that claims are supported. Controlled model and index failures produce explicit, recoverable states with no fabricated citations or results.

Stage 5 / Day 5: Validate physical outcomes

Status: **NOT STARTED** - project process and sensory tables have no measured trial records. RAG does not substitute for this physical work.

Prepare the comparison

- Select an A1 that satisfies the recipe and evidence gates. Confirm exact ingredient forms, supplier/batch identities and practical availability before preparation.
- Prepare B0, A0 and A1 using the frozen replacement rule. Keep A0/A1 algae dose and serving size equal. Record all intentional recipe differences so an observed change is not wrongly attributed to one ingredient.
- Record exact grams, batch size, process order, temperature, mixing time, yield, storage, preparation time and deviations. Bind process records to recipe IDs and versions.
- Create random sample codes and a protected mapping. Use consistent portions, serving temperature and a balanced tasting order. Complete the protocol before collecting ratings.

Collect and calculate

- Recruit 5-10 informed, voluntary adult participants for a directional pilot, subject to the project's applicable arrangements. Confirm relevant exclusions before offering samples.
- Record the chosen 1-9 scales and anchors for off-note intensity and liking. Capture one row per participant, sample and attribute using pseudonymous IDs.
- Keep the published studies' child panels, trained panels and alternative scales separate from this project trial.
- Calculate sample size, mean, spread and the prespecified A1-vs-A0 sensory comparison. Pair ratings within participant when the design supports it; report exclusions and deviations.
- Calculate B0/A0/A1 nutrition using recipe arithmetic. Label it calculated nutrition; laboratory composition requires its own measured records.
- Import measured summaries as the human-response modality. If used to alter candidate ranking, create a new version and keep the pre-trial run reproducible. Trial results must not silently change the frozen RAG corpus or held-out evaluation.

Files and deliverables

Populate prototype_processes.csv, project recipe records, sensory_trials.csv and sensory_summaries.csv. Keep personally identifying information and the protected blinding mapping outside the shared dataset. Add the recorded methods, deviations and outcomes to the run report.

Acceptance gate

B0/A0/A1 exist as prepared, traceable prototypes. The app shows calculated nutrition and a measured sensory comparison with sample size, scale, variability and limitations. Report neutral or adverse findings honestly. Completion depends on recording the result, not on achieving a positive effect.

If prototypes cannot be prepared, the software milestone may still be reported separately, but the original physical-validation requirement remains incomplete. Do not fill trial tables with simulated measurements.

Stage 6 / Day 6: Test and release

Status: **PARTIAL** - mock API/widget tests and report scaffolding exist. Real RAG evaluation, citation exports and release verification remain.

Report and export

- Generate the report from the stored run: goal, constraints, material, risk map, retrieved evidence, cited rationale, three variants where feasible, critique, B0/A0/A1, nutrition, sensory results, limitations and next experiment.
- Show calculated, measured, predicted, assumed and unknown values consistently. Include model/prompt and dataset/index versions in technical provenance.
- Support JSON export and a printable/PDF report. Citation links and locations must survive export. The same saved run must regenerate the same factual content after a restart.

Required verification

- Data: duplicate IDs, broken joins, missing provenance, units, permissions and invalid approval transitions.
- Retrieval: held-out positive questions, irrelevant/unsupported queries, species/tag filters, explicit matrix backoff, deduplication, source resolution and zero unapproved results.
- Index: absent/corrupt artifacts, model/revision/dimension mismatch and modified metadata or corpus hashes. Verify checks detect tampering or stale builds.
- Generation: malformed model output, invented citations, unsupported claims, retrieved text containing instructions, bounded retry/revision and provider outage.
- Formulation/impact: mass balance, exact ingredient form, bounds, allergens, infeasibility, missing nutrition, unit conversions and comparison formulas.
- API/Flutter: live run, error displays, evidence navigation, report export and restart persistence. Verify the real screen uses /runs and renders returned data.

Deployment and fallback

- Deploy the simplest reliable build with the pinned runtime, model files, dataset and index. Validate deployment memory/startup behavior and artifact integrity.
- Preload local embedding weights so retrieval does not depend on a model download during the demonstration. Document model licensing and reproduction steps.
- Prepare a clearly labelled saved run and local/offline retrieval build. If generation uses an online provider, an offline replay is a replay; demonstrate and label any deterministic fallback separately.
- Freeze features after critical issues are resolved. Any corpus change requires a new dataset/index release and retrieval re-evaluation; any prompt change requires affected generation checks.

Acceptance gate

The deployed build completes the fixed grounded run and one alternative. Required checks pass, no release-blocking defect remains, citations resolve, and reports regenerate from persisted data. The offline package opens the recorded evidence and report with its original versions. Measured results are included only if Stage 5 is complete.

Stage 7 / Day 7: Verify and hand off

Status: **NOT COMPLETE** - the grounded application, approved data/index release and physical-validation gates must be completed first.

Clean release check

- Start from a clean setup using the documented runtime and locked dependencies. Install or verify approved model artifacts, build/load the index and check readiness before running the fixed scenario.
- Re-run the held-out retrieval suite, one grounded generation and the alternative scenario. Confirm citations, recipe constraints, nutrition units, trial sample size and status labels.
- Record release commit, dataset/index/model identities, test results and known limitations. Make only release-blocking fixes and rerun affected checks.

Three-minute demonstration

- 0:00-0:25: describe the product, sensory problem, exact inputs and proof metric.
- 0:25-0:55: show the molecular/sensory evidence map and distinguish observed links from hypotheses.
- 0:55-1:30: show diagnosis/intervention queries, applied filters, ranked passages and source locations. Explain one indirect or excluded result.
- 1:30-2:10: run the six roles and show how retrieved evidence supports the selected candidate and cited explanation. Explain one rejected recipe or critique.
- 2:10-2:45: show B0/A0/A1, calculated protein density and measured sensory results if collected. State the sample size and relevant limitation.
- 2:45-3:00: state the outcome and next physical experiment. Keep the saved run and backup video available.

Final package

- Code and setup guide; exact model configuration and download instructions or permitted artifacts.
- Approved dataset, issue/exclusion log, review attribution and version manifest.
- Reproducible index build command, index artifacts where redistribution permits, and hashes.
- Retrieval evaluation cases/results, generation checks and application test results.
- Versioned demonstration run, recipe/process records, measured trial summaries when available, and exported impact report.
- Known limitations and task backlog; clear labels for unavailable features and incomplete physical evidence.

Final definition of done

RAG is complete only when approved evidence is retrieved for a real request, supplied to generation, used in supported cited output, and visible in the application with reproducible versions. The full sprint additionally requires valid formulations, traceable B0/A0/A1 physical records, the recorded sensory comparison, calculated nutrition, a tested release and an exportable report.

Reference and execution record

Reference: original sprint PDF, pp.18-23; repository scope, extraction review and current code. Status inspected 22 October 2026. Track gate owners, evidence and completion dates in docs/TASKS.md; structure validation, scientific approval, retrieval quality and physical trials each require verification.